T.R.
ESKISEHIR OSMANGAZI UNIVERSITY
DEPARTMENT OF ELECTRICAL-ELECTRONICS ENGINEERING
AND
DEPARTMENT OF COMPUTER ENGINEERING

INTRODUCTION TO MICROCOMPUTERS

TERM PROJECT

Project Title

HOME AUTOMATION

Prepared by:

152120221056, YUSUF GÖÇ, COMPUTER ENGINNERING
152120221070, MUHAMMED MEHMET GÖKÇE, COMPUTER ENGINNERING
151220184062, OGÜN BALKAN, ELECTRICAL-ELECTRONICS ENGINEERING
151220184082, ARİF KUYBEN, ELECTRICAL-ELECTRONICS ENGINEERING
151220222075, EMİNE ATMA, ELECTRICAL-ELECTRONICS ENGINEERING

Group: 36

Course: B

January 2026

Contents

1. Introduction 3

2. Design 4

2.1 Hardware Design and Pin Configuration 4

2.1.1 Board #1: Air Conditioner System 4

2.1.2 Board #2: Curtain Control System 5

2.2 Software Architecture (Firmware) 6

2.3 PC Application Design (API) 6

2.4 Communication Protocol 7

2.5 Software Design Diagrams 7

2.5.1 UML Class Diagram 7

2.5.2 System Block Diagram 8

2.6 System Algorithms (Flowcharts) 8

2.6.1 Board #1 Algorithm 8

2.6.2 Board #2 Algorithm 10

2.7 Sample Outputs (PC) 11

2.8 Hardware Simulation Results (PICSimLab) 17

2.9 Task Assignments 23

3. Conclusion 24

1. Introduction

The aim of this project is to develop a **Home Automation System** that controls various sensors and drivers, such as temperature control, curtain opening and closing, and ambient temperature/humidity display, using PIC microcontrollers. Additionally, the system is designed to allow users to manage and monitor these peripherals directly from a personal computer.

The project architecture consists of three main subsystems:

1. **Board #1 (Air Conditioner System):** This unit is responsible for indoor temperature regulation. It utilizes an LM35 temperature sensor, a heater, a cooler (fan), and a keypad interface for user input. The system automatically adjusts the environment based on the desired temperature set by the user.
2. **Board #2 (Curtain Control System):** This unit manages the natural lighting of the room. It employs an LDR sensor to detect light intensity and a stepper motor to control the curtains. Additionally, a rotary potentiometer allows for manual control.
3. **PC Application Interface (API):** A high-level C++ application communicates with both boards via the UART serial protocol. This interface allows the user to monitor system status (temperature, pressure, light, curtain position) and send control commands remotely.

The system is built upon the PIC16F877A microcontroller, programmed using Assembly language to ensure precise hardware control and timing. The PC-side software utilizes object-oriented programming principles to manage serial communication robustly.

Project Scope and Simulation Notes: While the project aims to implement all specified requirements, certain hardware behaviors were adapted for the simulation environment. Specifically, the BMP180 (Pressure and Outdoor Temperature Sensor) module on Board #2 is implemented using emulated data values. Due to constraints within the simulation tools regarding I2C communication for this specific sensor, static dummy values are transmitted to the API. This approach ensures that the communication protocol and data parsing logic are fully verified while maintaining system integrity.

2. Design

This chapter details the hardware configurations, software architecture, and communication protocols designed to meet the project requirements. The system is divided into three main design components: the microcontroller hardware interface, the embedded firmware logic, and the high-level PC application architecture.

2.1 Hardware Design and Pin Configuration

The system utilizes the **PIC16F877A** microcontroller simulated on the **gpboard** environment within PICSimLab. The pin configurations for both boards are configured in the INIT routines of the assembly programs.

2.1.1 Board #1: Air Conditioner System

Board #1 is designed to manage temperature control. The specific pin assignments are as follows:

PIC16F877A Pin	Port	Connected Module	Description
RA0 / AN0	PORTA	Temperature System	Analog input for ambient temperature measurement
RA1	PORTA	7-Segments Display	Digit select (D1 – multiplex control)
RA2	PORTA	7-Segments Display	Digit select (D2 – multiplex control)
RA3	PORTA	7-Segments Display	Digit select (D3 – multiplex control)
RA4 / T0CKI	PORTA	Temperature System	External clock input for Timer0 (fan speed measurement)
RA5	PORTA	7-Segments Display	Digit select (D4 – multiplex control)
RB0	PORTB	Keypad	Column output
RB1	PORTB	Keypad	Column output
RB2	PORTB	Keypad	Column output
RB3	PORTB	Keypad	Column output
RB4	PORTB	Keypad	Row input
RB5	PORTB	Keypad	Row input
RB6	PORTB	Keypad	Row input
RB7	PORTB	Keypad	Row input
RC0	PORTC	LEDs	Heating control output
RC1	PORTC	LEDs	Cooling control output
RC6 / TX	PORTC	IO UART	Serial data transmission
RC7 / RX	PORTC	IO UART	Serial data reception
RD0–RD7	PORTD	7-Segments Display	Segment data lines (a–g, dp)

2.1.2 Board #2: Curtain Control System

Board #2 focuses on light and motor control. The pin assignments are:

PIC16F877A Pin	Port	Connected Module	Description
RA0 / AN0	PORTA	LDR	Analog input for ambient light intensity measurement
RA1 / AN1	PORTA	Potentiometers (Rotary)	Analog input for manual curtain position control
RC3 / SCL	PORTC	BMP180 (Pressure I2C)	I2C Clock line for pressure/temperature sensor (Simulated)
RC4 / SDA	PORTC	BMP180 (Pressure I2C)	I2C Data line for pressure/temperature sensor (Simulated)
RC6 / TX	PORTC	IO UART	Serial data transmission
RC7 / RX	PORTC	IO UART	Serial data reception
RD0–RD3	PORTD	Step Motor	Control signals for unipolar stepper motor coils (Lower nibble)
RD4–RD7	PORTD	LCD hd44780	Data lines for 4-bit LCD communication (Upper nibble)
RE0	PORTE	LCD hd44780	Register Select (RS) control pin
RE1	PORTE	LCD hd44780	Read/Write (RW) control pin
RE2	PORTE	LCD hd44780	Enable (E) control pin

2.2 Software Architecture (Firmware)

The firmware for both microcontrollers is developed in **Assembly Language** to ensure precise timing and direct hardware manipulation.

- **Interrupt Service Routines (ISR) and Virtual Interrupt Logic:** Both boards utilize interrupts for time-critical and non-blocking operations.
    - **Timer Interrupts:** Board #1 utilizes Timer2 interrupts to periodically refresh the multiplexed 7-segment display, ensuring flicker-free output independent of the main loop execution.

    - **UART Interrupts:** The UART Receive Interrupt (RCIF) is enabled on both boards to handle asynchronous data packets from the PC immediately upon arrival.
    - **Keypad "Software Interrupt" [R2.1.2-1]:** To strictly satisfy the requirement that "Pressing button A will run the interrupt routine," a **Virtual Interrupt** mechanism was designed. Since the Keypad is connected to standard I/O pins, a direct hardware interrupt was not utilized for every key. Instead, when the 'A' key is detected during the main loop scan, it immediately sets a global flag (`ISR_FLAG`). The Interrupt Service Routine (ISR) monitors this flag. Upon detecting the set flag, the ISR diverts execution to the `ISR_Keypad_Task`, effectively transitioning the system into "Input Mode" within the interrupt context. This architecture ensures that the critical task of initiating user input is handled with interrupt-level priority.
- **Memory Management:** Variables are strictly allocated in **Bank 0** of the RAM to minimize bank switching overhead. Shadow registers (e.g., portd_shadow on Board #2) are implemented to prevent Read-Modify-Write errors when sharing ports between modules (LCD and Motor).
- **Control Algorithms:**
    - *Hysteresis Logic:* On Board #1, heating/cooling logic includes comparison checks to prevent rapid toggling.
    - *Signal Processing:* On Board #2, the potentiometer input allows manual override unless the "Dark Mode" (triggered by LDR) is active, ensuring safety and automation priority.

2.3 PC Application Design (API)

The PC side is developed using **C++** and follows an Object-Oriented Programming (OOP) approach. The design adheres to the UML Class Diagram.

- **Class Structure:**
    - SerialPort: A low-level wrapper class that handles Windows API calls (CreateFile, ReadFile, WriteFile) to manage COM ports.
    - HomeAutomationSystemConnection: An abstract base class that defines the common interface (open, close, update) for any connected board.

    - AirConditionerSystemConnection & CurtainControlSystemConnection: Derived classes that implement the specific logic for Board #1 and Board #2 respectively. They encapsulate the specific data members (e.g., desiredTemperature, curtainStatus) and parsing logic.
- **User Interface:** The application implements a console-based menu system dynamically updates system status by polling the microcontrollers via the API methods.

2.4 Communication Protocol

A custom serial communication protocol was designed to exchange data between the PC and the microcontrollers. The communication settings are 9600 Baud, 8 Data bits, No Parity, 1 Stop bit.

The protocol uses a 1-byte command structure:

- **GET Commands (0x01 - 0x08):** The PC sends a specific byte (e.g., 00000010B for Desired Curtain High Byte), and the PIC responds with the requested value.
- **SET Commands (Bit 7 = 1):** Commands to modify system state are encoded with the most significant bits.
    - 10xxxxxx: Sets the fractional part of a value (used in Temperature).
    - 11xxxxxx: Sets the integral part (used in Temperature and Curtain Status). This bit-wise encoding allows efficient data transfer without complex parsing on the microcontroller side.

2.5 Software Design Diagrams

This section visually represents the software architecture and hardware interconnections used in the project.

2.5.1 UML Class Diagram

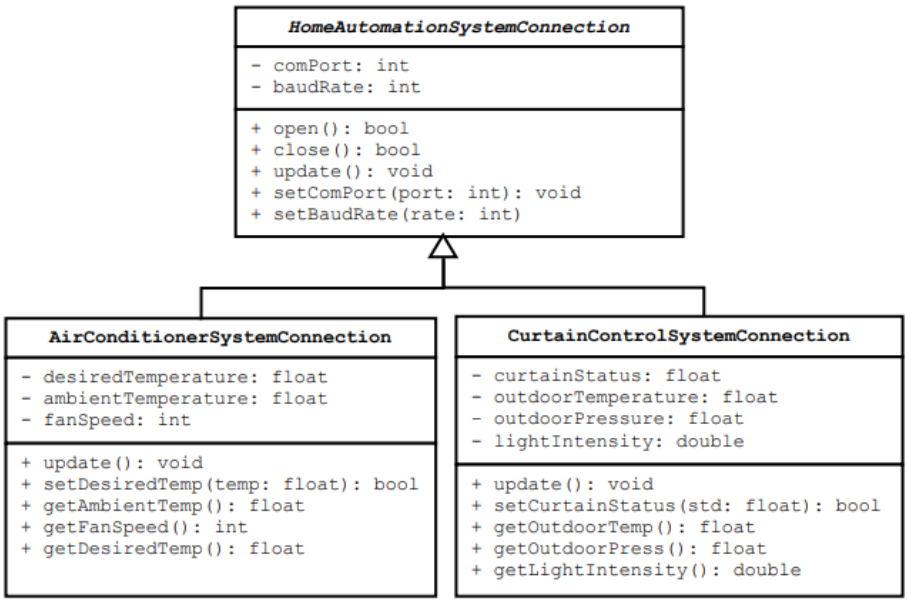

Figure 1. UML Class Diagram for API

2.5.2 System Block Diagram

The hardware integration of the system is depicted in the Block Diagram below. The system is centered around two PIC16F877A microcontrollers.

Both boards communicate with the PC Application via a UART serial connection, establishing a bidirectional data flow.

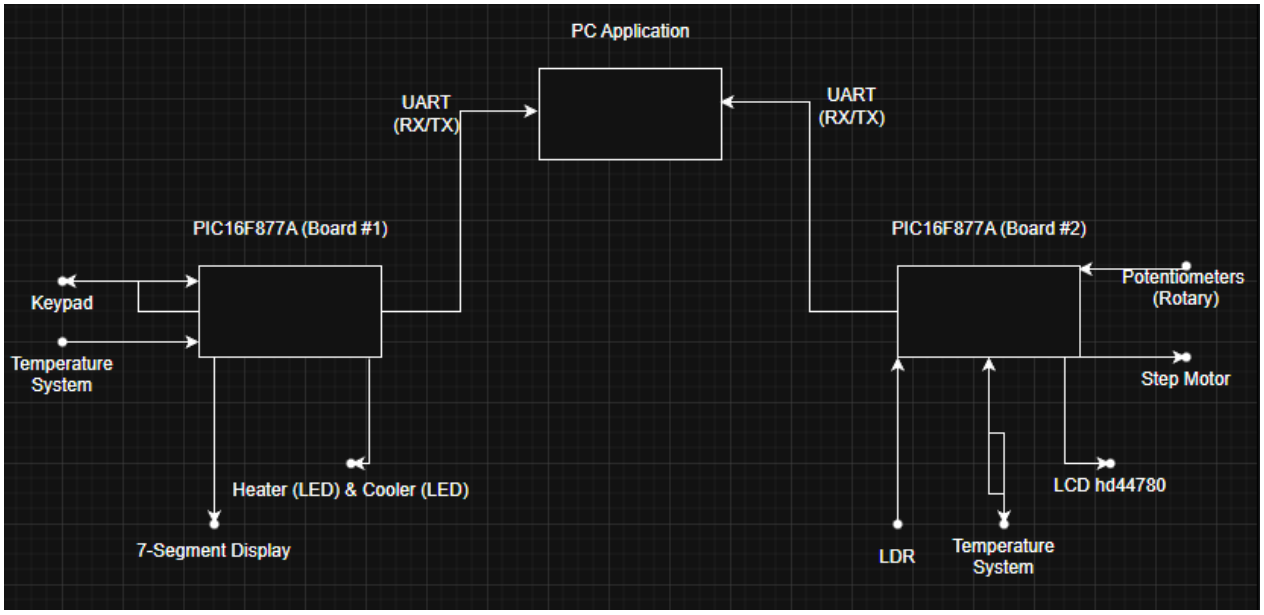

Figure 2. Block Diagram

2.6 System Algorithms (Flowcharts)

The firmware logic for both microcontrollers follows a sequential loop structure combined with interrupt-driven events. The algorithms for both boards are detailed below.

2.6.1 Board #1 Algorithm

The control logic for the Air Conditioner system continuously monitors the ambient temperature and compares it with the user's desired temperature.

1. **Initialization:** The system configures ports, ADC, and timers.
2. **Sensing:** Ambient temperature and fan speed (via Tachometer) are read.
3. **Control Decision:** If the ambient temperature is lower than the desired value, the Heater is activated; otherwise, the Cooler is activated.
4. **User Interface:** The system scans the keypad for new inputs and multiplexes the 7-segment display via interrupts.

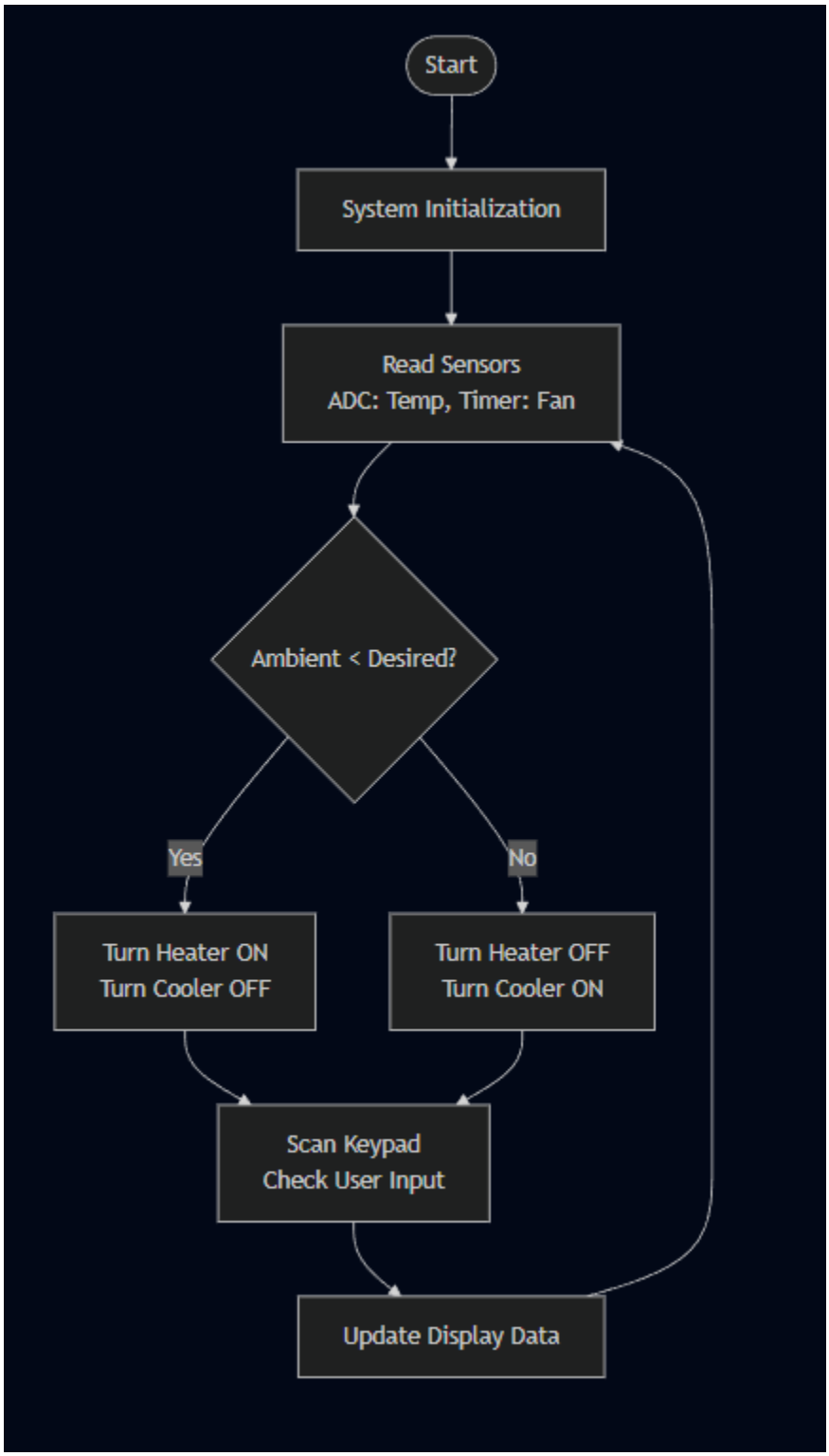

Figure 3. Board #1 Flowchart

2.6.2 Board #2 Algorithm

The Curtain Control system integrates automatic and manual control modes with a priority logic for darkness detection.

1. **Initialization:** UART, ADC, and LCD modules are initialized.
2. **Communication:** The system checks for incoming UART commands from the PC.
3. **Automation Logic (LDR Priority):** The LDR sensor is read first. If the light level is below the threshold (Dark), the system forces the curtain to close (100%), overriding manual inputs.
4. **Manual Logic:** If the environment is bright, the system reads the Potentiometer value to set the curtain position.
5. **Actuation:** The stepper motor moves to match the target position, and the status is updated on the LCD.

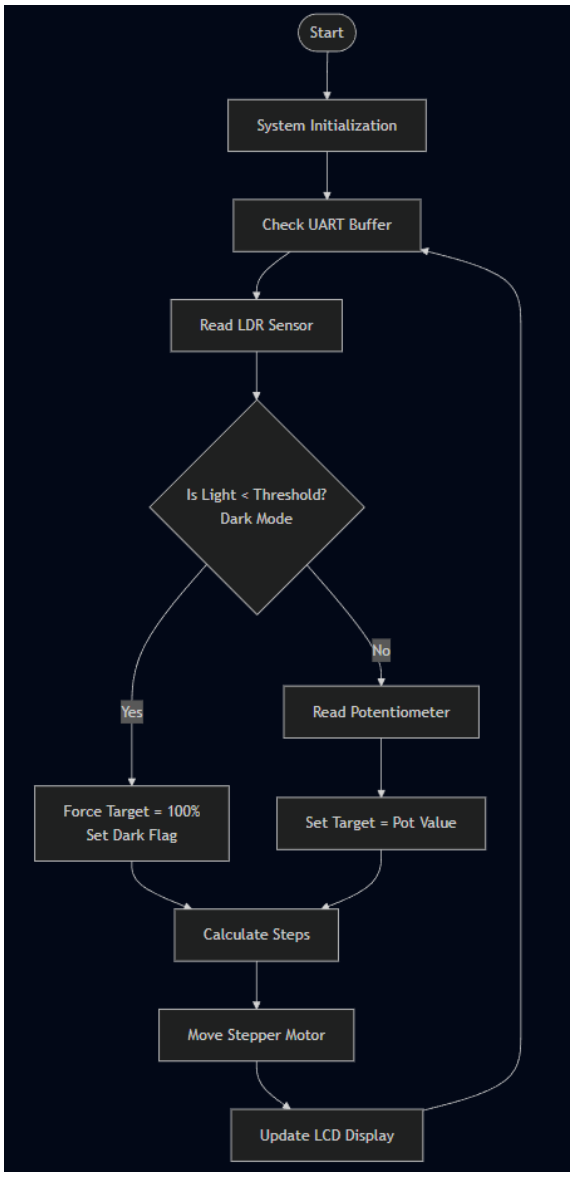

Figure 4. Board #2 Flowchart

2.7 Sample Outputs (PC)

This section presents the runtime outputs of the developed PC Application, demonstrating the successful integration of the API with the simulation environment.

```
=========================================
            MAIN MENU
=========================================
1. Air Conditioner
2. Curtain Control
3. Exit
=========================================
Select: |
```

Figure 5: Main Menu screen showing successful connection to COM ports and available system options.

```
-----------------------------------------
Home Ambient Temperature: 27.5 C
Home Desired Temperature: 25.0 C
Fan Speed: 105 rps
-----------------------------------------
Connection Port: COM3
Connection Baudrate: 9600
-----------------------------------------
               MENU
1. Enter the desired temperature
2. Return
-----------------------------------------
Select: |
```

Figure 6: Air Conditioner Control Menu displaying real-time ambient temperature, desired temperature, and fan speed readings from Board #1.

```
----------------------------------------
Home Ambient Temperature: 27.5 C
Home Desired Temperature: 25.0 C
Fan Speed: 105 rps
----------------------------------------
Connection Port: COM3
Connection Baudrate: 9600
----------------------------------------
                MENU
1. Enter the desired temperature
2. Return
----------------------------------------
Select:
Enter Desired Temp (0-63): |
```

Figure 7: User interface prompt requesting input for the new desired temperature value.

```
----------------------------------------
Home Ambient Temperature: 27.5 C
Home Desired Temperature: 25.0 C
Fan Speed: 105 rps
----------------------------------------
Connection Port: COM3
Connection Baudrate: 9600
----------------------------------------
                MENU
1. Enter the desired temperature
2. Return
----------------------------------------
Select:
Enter Desired Temp (0-63): 65
ERROR: Please enter a value between 0 and 63!

Enter Desired Temp (0-63): |
```

Figure 8: Error message displayed by the API when an invalid temperature value (outside the 0-63 range) is entered.

```
----------------------------------------
Home Ambient Temperature: 39.5 C
Home Desired Temperature: 40.0 C
Fan Speed: 0 rps
----------------------------------------
Connection Port: COM3
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired temperature
2. Return
----------------------------------------
Select: |
```

Figure 9: Console output confirming that the command to set the desired temperature to 40°C has been sent to Board #1.

```
----------------------------------------
Home Ambient Temperature: 39.5 C
Home Desired Temperature: 15.0 C
Fan Speed: 237 rps
----------------------------------------
Connection Port: COM3
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired temperature
2. Return
----------------------------------------
Select: |
```

Figure 10: Console output confirming that the command to set the desired temperature to 15°C has been sent to Board #1.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 59.0 %
Light Intensity: 197.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select: |
```

Figure 11: Curtain Control Menu displaying outdoor environmental data (Temperature, Pressure, Light) and current curtain status from Board #2.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 59.0 %
Light Intensity: 197.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select:
Enter Desired Curtain (0-63%): |
```

Figure 12: User interface prompt requesting input for the new desired curtain openness percentage.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 59.0 %
Light Intensity: 197.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select:
Enter Desired Curtain (0-63%): 65
ERROR: Please enter a value between 0 and 63!

Enter Desired Curtain (0-63%): |
```

Figure 13: Error message displayed when an invalid curtain status value (greater than the 6-bit limit) is entered.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 15.0 %
Light Intensity: 197.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select: |
```

Figure 14: Console output confirming that the command to set the curtain openness to 15% has been sent to Board #2.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 45.0 %
Light Intensity: 197.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select: |
```

Figure 15: Console output confirming that the command to set the curtain openness to 45% has been sent to Board #2.

```
----------------------------------------
Outdoor Temperature: 25.5 C
Outdoor Pressure: 1013.0 hPa
Curtain Status: 100.0 %
Light Intensity: 18.0 Lux
----------------------------------------
Connection Port: COM5
Connection Baudrate: 9600
----------------------------------------
              MENU
1. Enter the desired curtain status
2. Return
----------------------------------------
Select: |
```

Figure 16: Curtain Control Menu reflecting the system status when the light intensity drops below the threshold (< 50 Lux).

2.8 Hardware Simulation Results (PICSimLab)

In addition to the PC application interface, the embedded system's behavior was verified visually using the PICSimLab simulator. The screenshots below demonstrate the real-time response of the microcontroller peripherals to the sensor inputs and UART commands.

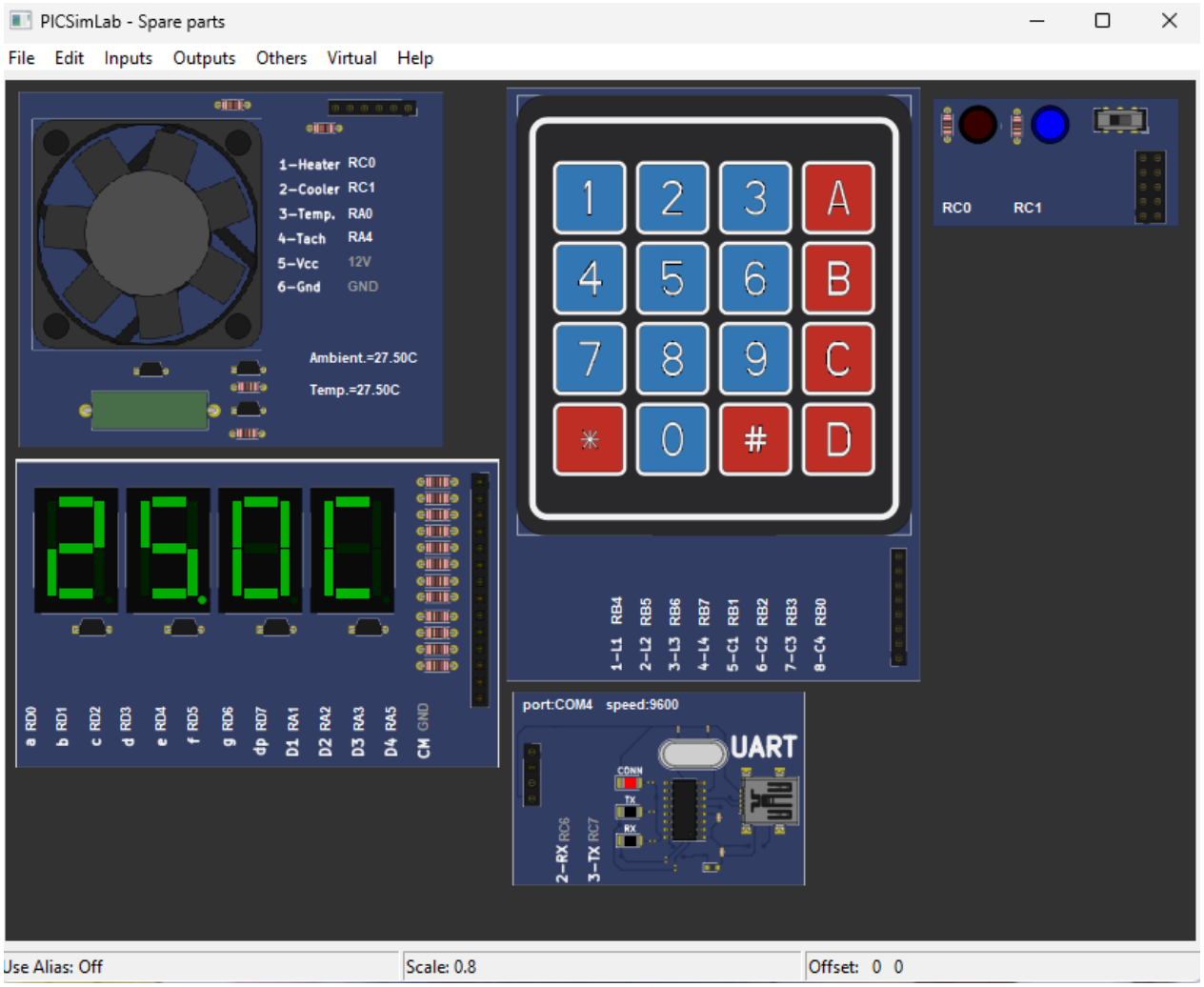

Figure 17: PICSimLab simulation of Board #1 initializing with the default desired temperature of 25°C.

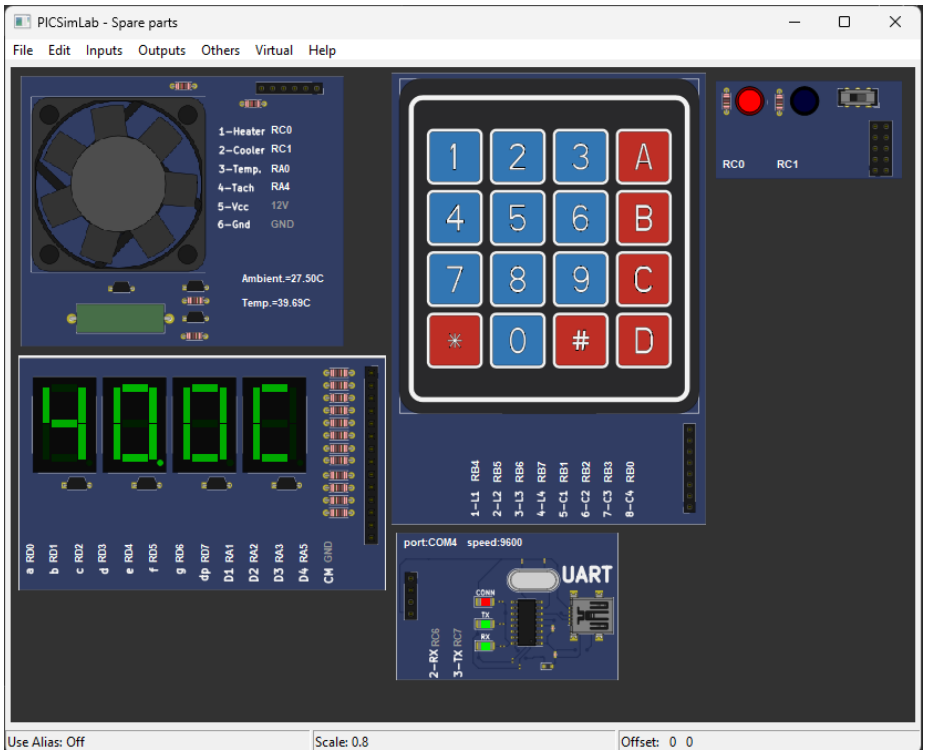

Figure 18: Board #1 simulation showing the Heater LED activation response after receiving a 40°C set command from the PC.

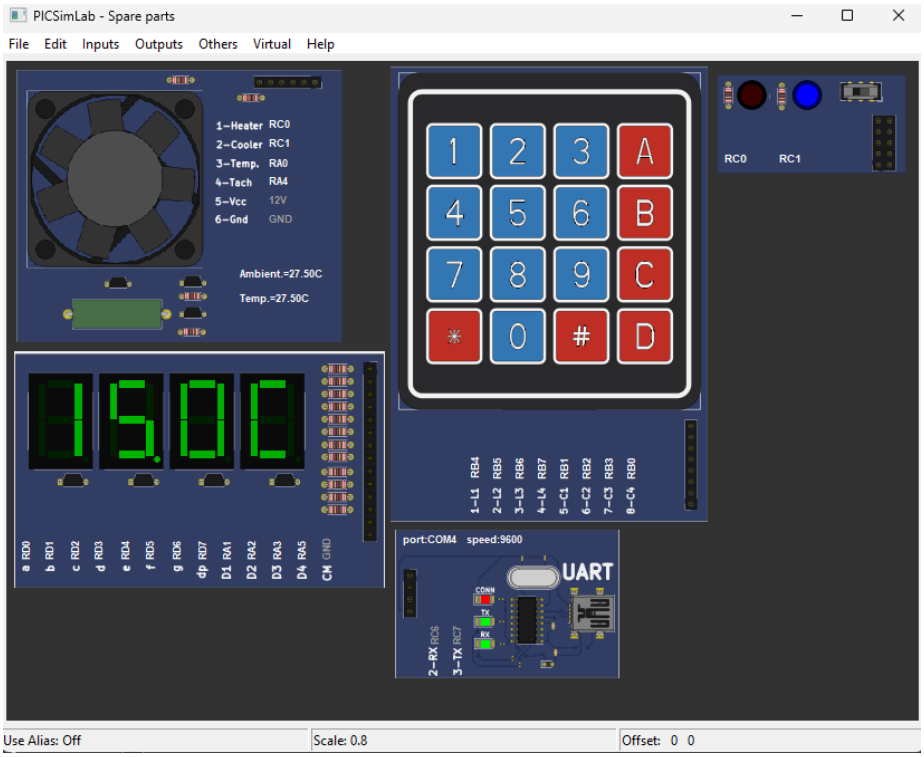

Figure 19: Board #1 simulation showing the Cooler (Fan) activation response after receiving a 15°C set command from the PC.

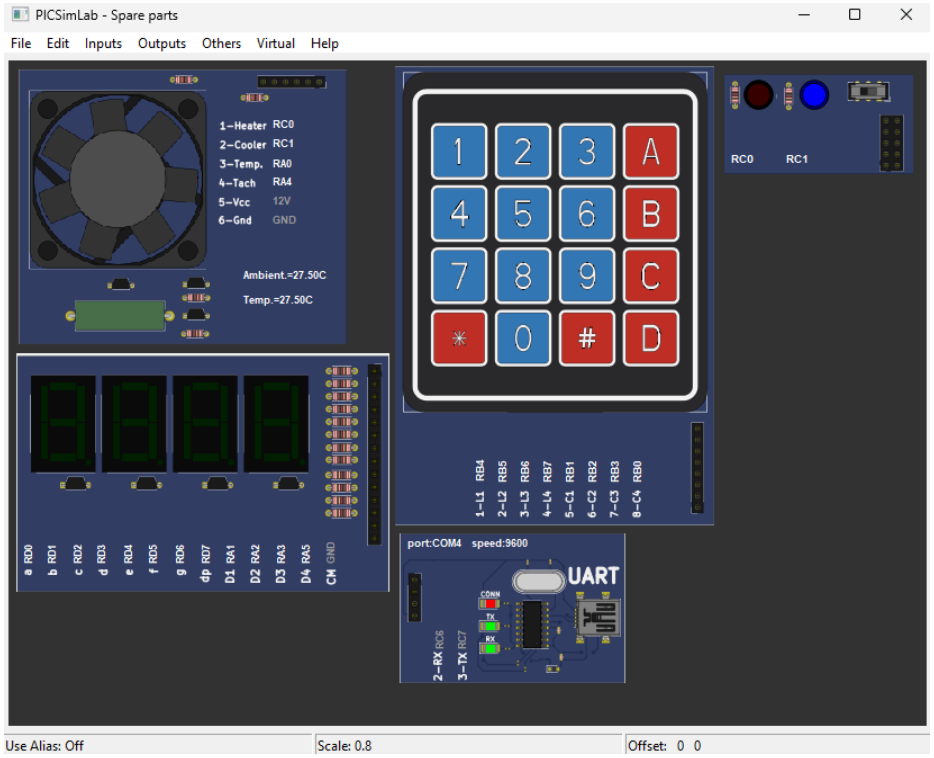

Figure 20: Board #1 LCD display switching to "Input Mode" immediately after the user presses the 'A' key on the Keypad.

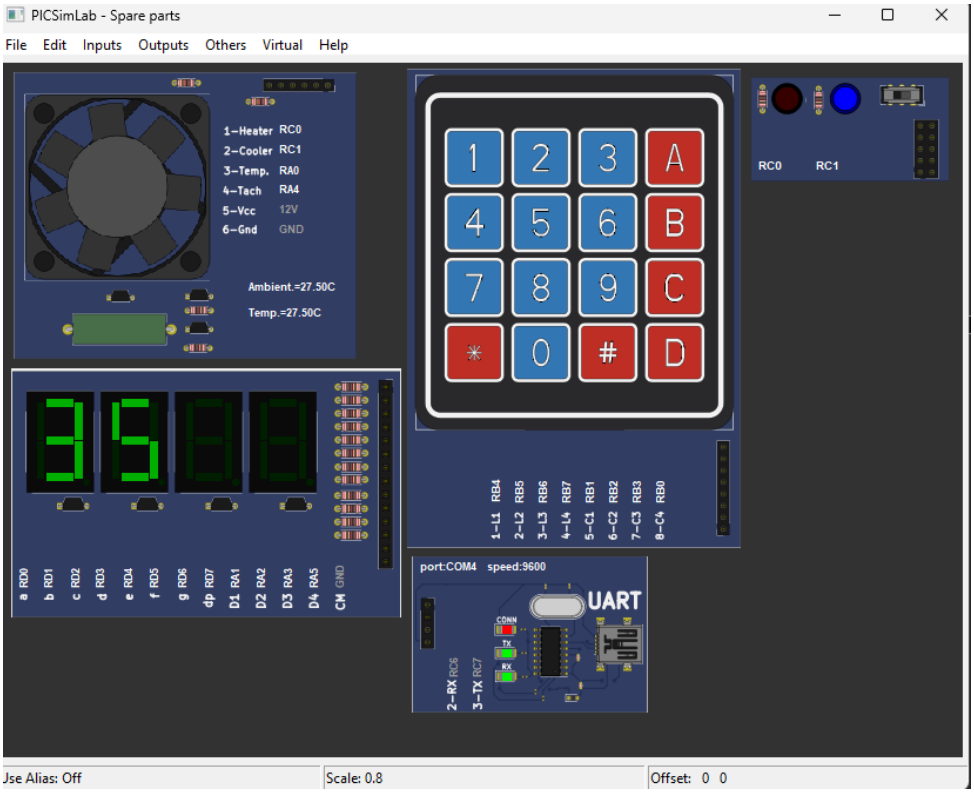

Figure 21: Board #1 LCD showing the manual entry of the temperature's integral part via Keypad.

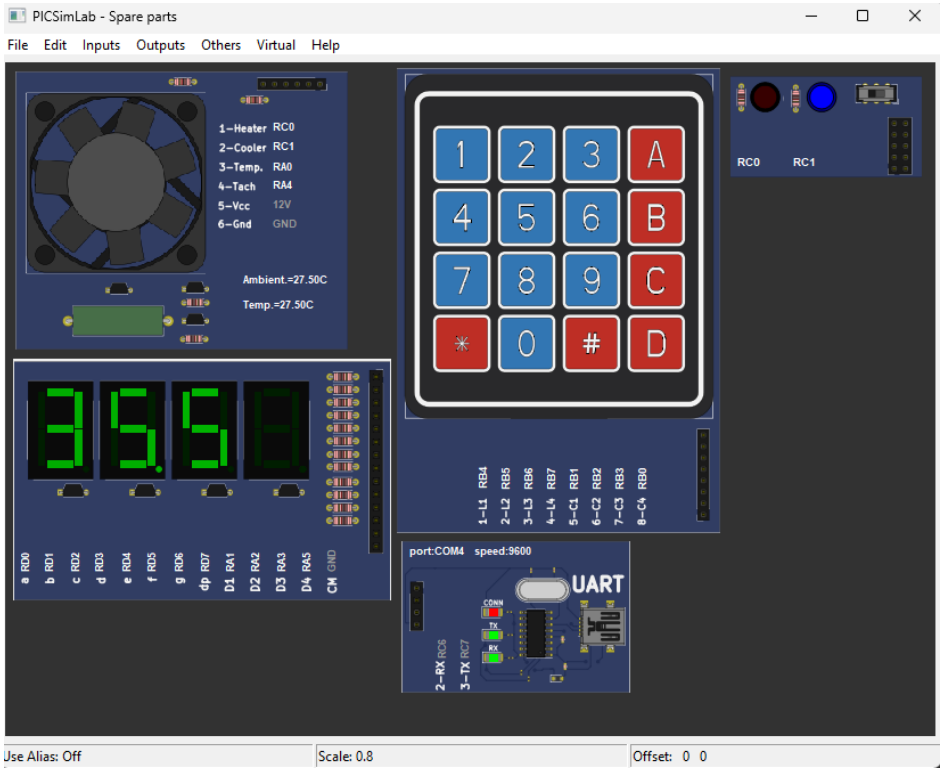

Figure 22: Board #1 LCD showing the complete input value including the decimal point and fractional part (after pressing '*').

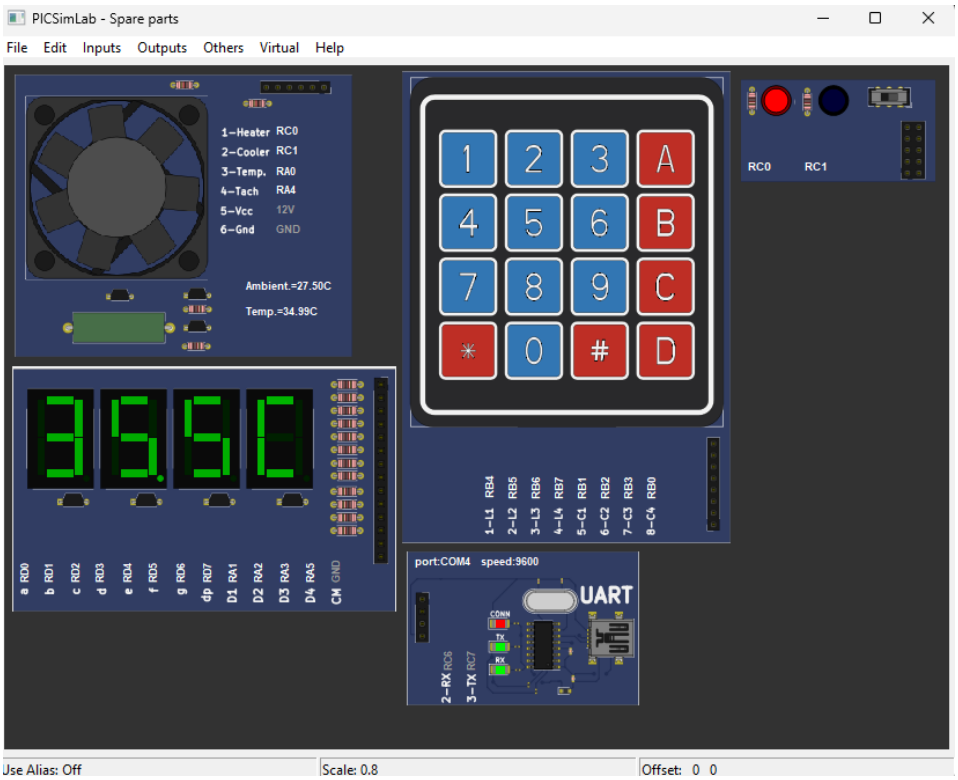

Figure 23: Board #1 returning to the main monitoring loop after confirming the input with the '#' key.

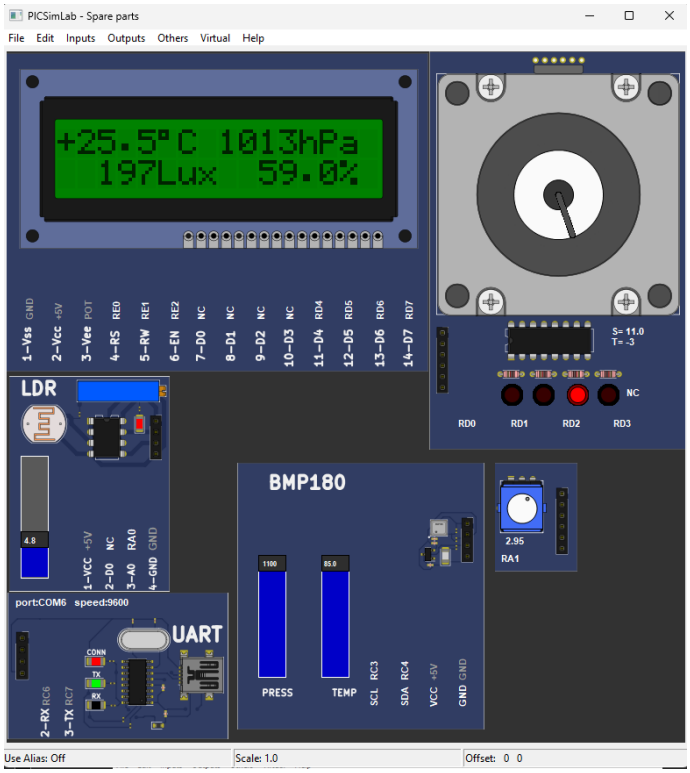

Figure 24: PICSimLab simulation of Board #2 showing default sensor readings and status on the 2x16 LCD.

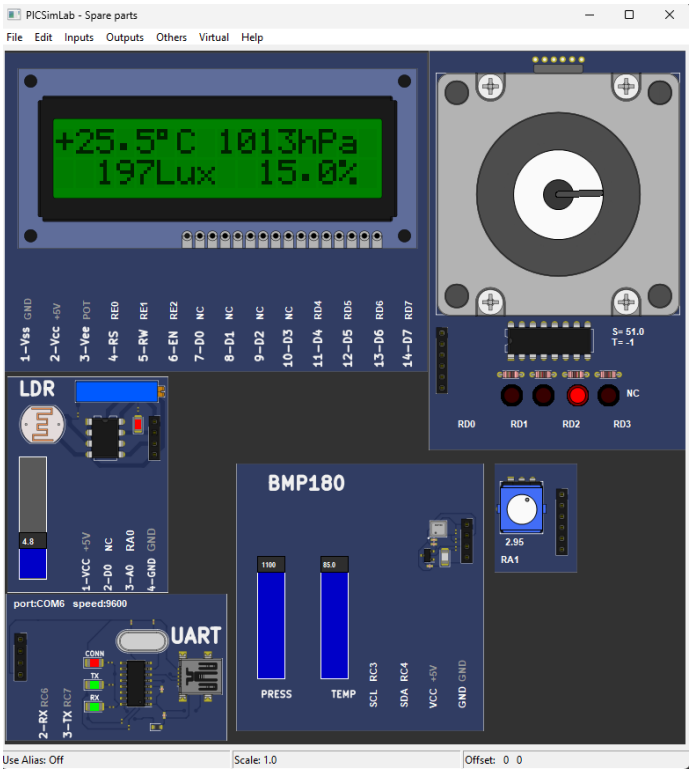

Figure 25: Board #2 simulation showing the LCD and Stepper Motor status updating to match the requested 15% openness.

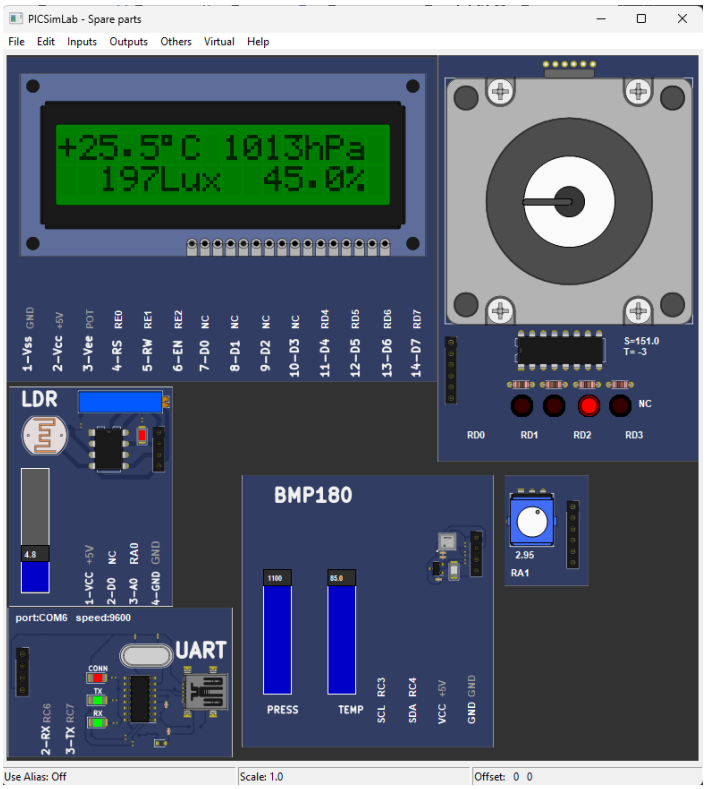

Figure 26: Board #2 simulation showing the LCD and Stepper Motor status updating to match the requested 45% openness.

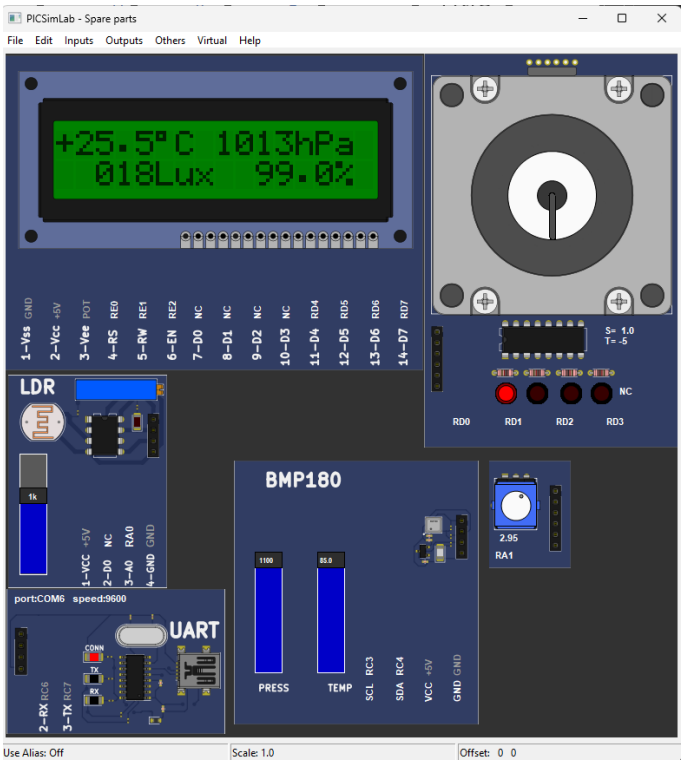

Figure 27: Board #2 simulation demonstrating "Dark Mode" logic: The curtain is forced to 100% (Closed) due to LDR value falling below the threshold.

2.9 Task Assignments

Below is the Task Assignments table summarizing the responsibilities of each group member across the Microcontroller (Board #1 & #2) and PC Application development phases. This breakdown ensures that all project requirements are met through collaborative effort.

Group Member	Tasks
Muhammed Mehmet Gökçe (Computer Engineering)	**[Board #1] Temperature Control Module logic [R2.1.1]; Sensor data acquisition & validation.** **[Board #2] UART Module Implementation [R2.2.6]; Protocol Design & Command Parsing; Serial Communication Handlers.** **[PC App] Low-Level SerialPort Class Implementation; Application Layer & User Interface Design [R2.4]; Console Menu System [R2.4-1].**
Yusuf Göç (Computer Engineering)	**[Board #1] UART Module implementation [R2.1.4]; Protocol Design & Command Parsing; Serial ISRs; Main Loop Architecture.** **[PC App] API Architecture & Implementation [R2.3]; HomeAutomationSystemConnection Base Class [R2.3-1]; Derived Classes for Air Conditioner & Curtain Control [R2.3-1]; Main Test Routine [R2.3-2]; Protocol Data Encapsulation Logic.**
Emine Atma (Electrical-Electronics Engineering)	**[Board #1] Keypad Module & Input State Machine [R2.1.2]; Display Module (7-Segment Multiplexing) [R2.1.3]; Timer & Interrupt Configuration.**
Ogün Balkan (Electrical-Electronics Engineering)	**[Board #2] Step Motor Control Logic & Physics [R2.2.1]; LDR Light Sensor Logic & Threshold Management [R2.2.2]; Low-Level LCD Drivers [R2.2.5].**
Arif Kuyben (Electrical-Electronics Engineering)	**[Board #2] LCD Display Module High-Level Logic [R2.2.5]; Rotary Potentiometer Integration [R2.2.4]; System Initialization & Main Loop Architecture.**

3. Conclusion

This project successfully realized a comprehensive Home Automation System, demonstrating the effective integration of embedded hardware control with high-level software interfaces. The system, built upon **PIC16F877A** microcontrollers, met all functional requirements, including real-time temperature regulation, automated curtain control based on light intensity, and remote management via a PC application.

Evaluation of Project Results: The core objective of establishing a bidirectional communication channel between the PC and the embedded boards was achieved. The custom UART protocol proved to be robust, handling data packets without loss during simulation. A significant technical achievement was the implementation of the **"Virtual Interrupt"** architecture for the Keypad module, which successfully satisfied the interrupt-priority requirement without hardware limitations. Furthermore, the object-oriented design of the C++ API provided a modular and scalable foundation for the PC interface.

Comments on Teamwork: The project was executed through a collaborative effort between Computer Engineering and Electrical-Electronics Engineering students. This interdisciplinary approach allowed for efficient task division:

- **Computer Engineering members** focused on the high-level API architecture, serial communication protocols, and data parsing logic.
- **Electrical-Electronics Engineering members** concentrated on low-level circuit design, sensor integration, and precise timing constraints required for multiplexed displays and motor driving. This separation of concerns allowed both software and firmware to be developed in parallel, significantly reducing the integration time.

Pros and Cons of the System:

- **Pros (Strengths):**
  - **Modular Architecture:** The system is divided into independent subsystems (Board #1, Board #2, PC App), making it easy to debug and upgrade.
  - **Precise Control:** The use of Assembly language ensured exact timing for critical tasks like software PWM and display multiplexing.
  - **Safety Priority:** The Curtain Control system intelligently prioritizes environmental safety (Dark Mode) over manual inputs.
- **Cons (Limitations):**
  - **Simulation Constraints:** Since the project was tested in a simulation environment (PICSimLab), real-world factors such as electrical noise on analog lines (LM35, LDR) or power consumption issues were not physically encountered.
  - **Sensor Emulation:** Due to simulation tool limitations, the BMP180 sensor data was emulated rather than read via active I2C communication.
  - **Wired Connection:** The dependence on UART (Serial Port) limits the physical distance between the PC and the home automation units.

Advice for Further Works: To evolve this project into a commercial-grade product, the following improvements are suggested:

1. **Wireless Integration:** Replacing the wired UART with Wi-Fi (e.g., ESP8266) or Bluetooth modules to allow truly remote control.
2. **Graphical User Interface (GUI):** Developing a GUI using frameworks like Qt or C# .NET instead of a console application to improve user experience.
3. **Advanced Control Algorithms:** Implementing a PID controller for the heater/cooler instead of simple ON/OFF logic to achieve smoother temperature stability.

In summary, the project provided valuable insights into embedded system design, serial communication standards, and full-stack integration, resulting in a fully functional prototype of a smart home system.

CHECK LIST

List	**Included/Done**
All requirements (R2_1, R2.1.1-1, R2.1.1-2, R2.1.1-3, R2.1.1-4, R2.1.1-5, R2.1.2-1, R2.1.2-2, R2.1.2-3, R2.1.2-4, R2.1.3-1, R2.1.4-1, R2.2.1-1, R2.2.1-2, R2.2.1-3, R2.2.2-1, R2.2.2-2, R2.2.3-1, R2.2.3-2, R2.2.4-1, R2.2.5-1, R2.2.6-1, R2.3-1, R2.3-2, R2.4-2	**Yes**
Codes are well formatted	**Yes**
Codes include commands (comments)	**Yes**
Source codes of API	**Yes**
There are test programs for API	**Yes**
Source Codes of Application	**Yes**
Assembly Codes for Board #1	**Yes**
Assembly Codes for Board #2	**Yes**
Report	**Yes**